

7、cartographer建图算法

该章节需要搭配小车底盘以及其它传感器才能运行，这里只是说明实现的方式，实际上运行不了，需要搭配本公司的RDK-X3旭日派小车实现该功能。要移植到自己的主板上运行，还需安装其它依赖，因此，这里只是做代码演示，望须知。

7.1、程序功能说明

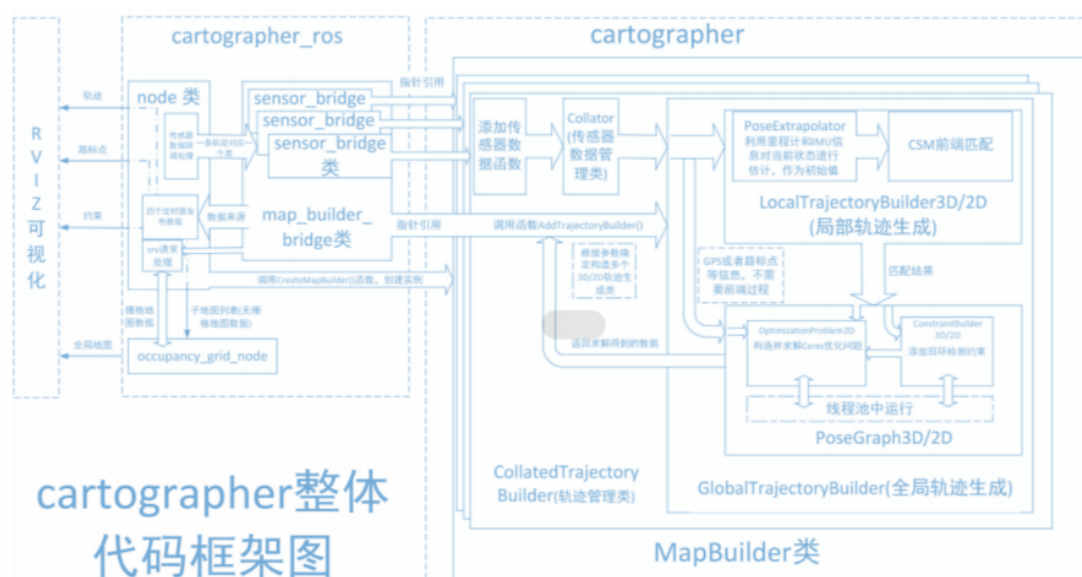
程序启动后，通过手柄或键盘控制小车，小车将在移动的过程中利用雷达扫描数据，使用cartographer算法构建地图，构建完成后保存地图。

7.2、Cartographer简介

Cartographer ROS2: https://github.com/ros2/cartographer_ros

- cartographer

Cartographer是Google开源的一个ROS系统支持的2D和3D SLAM（simultaneous localization and mapping）库。基于图优化（多线程后端优化、ceres构建的problem优化）的方法建图算法。可以结合来自多个传感器（比如，LIDAR、IMU 和 摄像头）的数据，同步计算传感器的位置并绘制传感器周围的环境。cartographer的源码主要包括三个部分：cartographer、cartographer_ros和ceres-solver（后端优化）。



cartographer采用的是主流的SLAM框架，也就是特征提取、闭环检测、后端优化的三段式。由一定数量的LaserScan组成一个submap子图，一系列的submap子图构成了全局地图。用LaserScan构建submap的短时间过程累计误差不大，但是用submap构建全局地图的长时间过程就会存在很大的累计误差，所以需要利用闭环检测来修正这些submap的位置，闭环检测的基本单元是submap，闭环检测采用scan_match策略。cartographer的重点内容就是融合多传感器数据（odometry、IMU、LaserScan等）的submap子图创建以及用于闭环检测的scan_match策略的实现。

- cartographer_ros

cartographer_ros是在ROS下面运行的，可以以ROS消息的方式接受各种传感器数据，在处理过后又以消息的形式publish出去，便于调试和可视化。

7.3、安装cartographer

以ros2-foxy为例，输入以下指令安装，

```
sudo apt install ros-foxy-cartographer* -y
```

7.4、代码路径

launch源码，该功能源码的位置位于配套虚拟机中，

```
~/oradar_ws/src/yahboomcar_nav/launch
```

7.5、程序启动

7.5.1、建图

终端输入，

```
ros2 launch yahboomcar_nav map_cartographer_launch.py
```

7.5.2、地图保存

终端输入，

```
ros2 launch yahboomcar_nav save_map_launch.py
```

地图保存路径如下：

```
~/oradar_ws/src/yahboomcar_nav/maps
```

包括一个.pgm图片和一个.yaml文件。其中，.yaml文件内容如下：

```
image: ~/oradar_ws/src/yahboomcar_nav/maps/yahboom_map.pgm
mode: trinary
resolution: 0.05
origin: [-5.95, -3.26, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

参数解析：

- image：地图文件的路径，可以是绝对路径，也可以是相对路径
- mode：该属性可以是trinary、scale或者raw之一，取决于所选择的mode，trinary模式是默认模式
- resolution：地图的分辨率，米/像素
- origin：地图左下角的 2D 位姿(x,y,yaw), 这里的yaw是逆时针方向旋转的（yaw=0 表示没有旋转）。目前系统中的很多部分会忽略yaw值。
- negate：是否颠倒 白/黑、自由/占用的意义（阈值的解释不受影响）
- occupied_thresh：占用概率大于这个阈值的像素，会被认为是完全占用。
- free_thresh：占用概率小于这个阈值的像素，会被认为是完全自由。

7.6、核心代码解析

map_cartographer_launch.py

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource

def generate_launch_description():

    laser_bringup_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('yahboomcar_nav'), 'launch'),
            '/laser_bringup_launch.py'
        ])
    )

    cartographer_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('yahboomcar_nav'), 'launch'),
            '/cartographer_launch.py'
        ])
    )

    return LaunchDescription([laser_bringup_launch, cartographer_launch])
```

laser_bringup_gmapping_launch.py启动雷达底盘， cartographer_launch.py启动cartographer建图相关节点。

cartographer_launch

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch_ros.actions import Node
from launch.substitutions import LaunchConfiguration
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import ThisLaunchFileDir

def generate_launch_description():
    use_sim_time = LaunchConfiguration('use_sim_time', default='false')
    package_path = get_package_share_directory('yahboomcar_nav')
    configuration_directory = LaunchConfiguration('configuration_directory',
    default=os.path.join(
                                                package_path, 'params'))
    configuration_basename = LaunchConfiguration('configuration_basename',
    default='cartographer_config.lua')

    resolution = LaunchConfiguration('resolution', default='0.05')
    publish_period_sec = LaunchConfiguration(
```

```

        'publish_period_sec', default='1.0')

    return LaunchDescription([
        DeclareLaunchArgument(
            'configuration_directory',
            default_value=configuration_directory,
            description='Full path to config file to load'),
        DeclareLaunchArgument(
            'configuration_basename',
            default_value=configuration_basename,
            description='Name of lua file for cartographer'),
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='false',
            description='Use simulation (Gazebo) clock if true'),

        Node(
            package='cartographer_ros',
            executable='cartographer_node',
            name='cartographer_node',
            output='screen',
            parameters=[{'use_sim_time': use_sim_time}],
            arguments=['-configuration_directory', configuration_directory,
                      '-configuration_basename', configuration_basename]),

        DeclareLaunchArgument(
            'resolution',
            default_value=resolution,
            description='Resolution of a grid cell in the published occupancy
grid'),

        DeclareLaunchArgument(
            'publish_period_sec',
            default_value=publish_period_sec,
            description='OccupancyGrid publishing period'),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                [ThisLaunchFileDir(), '/occupancy_grid_launch.py']),
            launch_arguments={'use_sim_time': use_sim_time, 'resolution':
resolution,
                             'publish_period_sec': publish_period_sec}.items(),
            ),
    ])

```